# Install guide

Three parts to install, then a verification step. Budget ~15 minutes the first time. Read the **Common snags** at the bottom if anything misbehaves — every issue listed there is one a real user actually hit.

```
Part A — the Fusion add-in (the listener)
Part B — Python + dependencies
Part C — register with Claude Desktop
Verify  — preflight check, then ping from Claude
```

## Part A — Install the Fusion add-in

The listener must be an **Add-In**, not a Script. (Scripts run once and exit; add-ins stay loaded — which the listener needs.) This trips people up, so follow the type carefully.

1. Open Fusion 360.
2. Press **Shift+S** (or **Utilities → Add-Ins → Scripts and Add-Ins**).
3. Click the **Add-Ins** tab (not Scripts).
4. Click the green **+** to create. In the dialog choose **Type: Add-In**, **Language: Python**, **Name:** `MCPListener`, then Create. - This creates a folder at:
   - **Windows:** `%appdata%\Autodesk\Autodesk Fusion 360\API\AddIns\MCPListener\`
   - **macOS:** `~/Library/Application Support/Autodesk/Autodesk Fusion 360/API/AddIns/MCPListener/`
5. Select `MCPListener` in the list, click the **pencil / Edit** to open its folder, and **replace the generated** `MCPListener.py` with the one from this package (same filename).
6. Back in the Add-Ins tab, select `MCPListener`, tick **Run on Startup**, and click **Run** (▶). A popup should say *"MCPListener running on http://127.0.0.1:8765"*.

> **The AddIns folder is the only copy Fusion runs.** If you keep the project files elsewhere (a repo/working folder), updating that copy does NOT update the add-in - you must copy the new `MCPListener.py` into the AddIns folder above and restart the add-in (Stop › Run). A stale add-in with a newer server now reports PROTOCOL MISMATCH instead of failing silently.
>
> Leave Fusion open while using Claude. With "Run on Startup" ticked, the listener auto-starts each time Fusion launches.

**If you accidentally made it a Script** (no "Run on Startup" toggle appears for it, and its path contains `\API\Scripts\`): delete/ignore that one and recreate it under the **Add-Ins** tab as in step 4. The file can't just be moved — add-ins and scripts are registered differently.

---

## Part B — Python and dependencies

1. Have **Python 3.10+** installed and on your PATH. Check:
   `bash python --version # or: python3 --version`
2. Put this package folder somewhere stable, e.g. `C:\Users\<you>\fusion-mcp\` or `~/fusion-mcp/`.
3. Install dependencies **into the interpreter you'll point Claude at**. The safest way is the `-m pip` form so it can't land in the wrong Python: `bash python -m pip install -r requirements.txt` (or `python3 -m pip install -r requirements.txt`)

> **Why `-m pip`?** If you have several Pythons, a bare `pip install` can install into a different one than the launcher/Claude uses, and you'll get "No module named 'requests'" or "No module named 'mcp'" even though pip said success. `python -m pip` installs into *that* python, every time.

---

## Part C — Register with Claude Desktop

1. Open **Claude Desktop → Settings → Developer → Edit Config**. **Always reach the file this way** — it opens the exact file Claude Desktop reads (`claude_desktop_config.json`). Don't go hunting in folders; the location varies and editing the wrong file is the #1 reason "nothing happens."
2. Add a `fusion360` server. If the file is empty, make it exactly:

**Windows:** `json { "mcpServers": { "fusion360": { "command": "python", "args": ["C:/Users/<you>/fusion-mcp/fusion_mcp_server.py"] } } }`

**macOS / Linux:** `json { "mcpServers": { "fusion360": { "command": "python3", "args": ["/Users/<you>/fusion-mcp/fusion_mcp_server.py"] } } }`

- Use an **absolute** path with **forward slashes** (or doubled `\\` on Windows — never single backslashes).
- If the config already has `mcpServers` with other entries, **add `fusion360` as another key inside it** — don't paste a second `mcpServers` block or a second top-level `{ }`. 3. **Fully quit and reopen** Claude Desktop (not just close the window).

---

## Verify

1. With Fusion open (listener running) and Claude Desktop restarted, run the preflight from this folder: - **Windows:** double-click `launch.bat`, or `launch.bat check` - **macOS/Linux:** `./launch.sh` (run `chmod +x launch.sh` once first)

Everything should read **OK**. If `requests` or `mcp` shows FAIL, redo the `python -m pip install` with the right interpreter. 2. In Claude Desktop, confirm the `fusion360` tools appear (tools/connector icon near the chat box), then ask:

> **Ping Fusion.**

A reply of **"alive"** confirms the whole chain: Claude → server → listener. 3. First model:

> **Make a 20 mm cube and show me.**

---

## Is the server running? (a common confusion)

You **don't** start `fusion_mcp_server.py` yourself, and the launcher does **not** keep it running. Here's the model:

- **Claude Desktop launches the server** automatically (as a hidden background child process) whenever it starts, based on the config. It stops when you quit Claude Desktop.
- **The launcher only inspects** your setup — it never leaves anything running. When `launch.bat` says "press any key" and the window closes, nothing was started or stopped.
- **To know it's running:** ask Claude to "Ping Fusion." If the `fusion360` tools show up and the ping works, it's running. If the tools never appear, Claude Desktop couldn't launch it — check the config path and the interpreter.

The only time you'd run the server in a terminal is debugging: `python launch.py serve` (used with the MCP Inspector). Not needed normally.

---

## Common snags

**"No module named 'requests'" (or 'mcp') even after pip installed it.** You installed into a different Python than the launcher/Claude uses. Re-run with `python -m pip install -r requirements.txt` using the *same* command ( `python` vs `python3` ) that your Claude config's `command` uses. The preflight prints which interpreter it's using — match that.

**Preflight says "Claude Desktop config not found."** The launcher checks the standard location ( `%APPDATA%\Claude\claude_desktop_config.json` / `~/Library/Application Support/Claude/...` ).

If yours is elsewhere, that's fine — trust **Settings → Developer → Edit Config** as the source of truth instead. The WARN doesn't block anything.

**Tools don't appear in Claude Desktop.** Almost always the config: invalid JSON (a stray/missing comma), a relative or wrong path, or the wrong interpreter. Validate the JSON, use an absolute path, fully restart Claude Desktop. A quick JSON sanity check:

```
python -m json.tool < "path/to/claude_desktop_config.json"
```

If that errors, your JSON is malformed.

**"Connection error" when a tool runs, but the server is fine.** That's the *listener* — Fusion isn't open, or the add-in isn't running. Shift+S → Add-Ins → MCPListener → Run. Then "Ping Fusion" again.

**Python 3.13 / 3.14.** The SDK generally imports on very new Pythons, but they run ahead of what some dependencies expect. If the server misbehaves in ways the config can't explain, install a 3.11 or 3.12 alongside and point the Claude config `command` at that interpreter's full path. This is the first thing to try for weird, hard-to-place errors.

**OneDrive "doubled" paths.** Some Windows setups have paths like `C:\Users\<you>\Documents\Documents\...` (OneDrive redirection — the doubled `Documents` is real). Whatever your actual path is, make the Claude config match it character-for-character.

**Geometry comes out 10× too big or small.** A units slip somewhere in custom code. All tool arguments are millimeters; Fusion's API is centimeters. Ask Claude to verify with `get_model_state` or `measure_distance`.

**Save / open / list-designs tools error.** Those need you signed into Fusion with an **active project** selected in the Data Panel. They don't work offline or on a brand-new unsaved doc.